

JavaScript Programming Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Course Overview

This JavaScript Programming course is designed to build a strong foundation in modern JavaScript for web development, frontend development, backend development, automation, and full-stack application development.

The course starts with programming fundamentals and JavaScript syntax, then progresses through functions, arrays, objects, DOM manipulation, events, browser APIs, ES6+ features, object-oriented programming, asynchronous programming, promises, async/await, modules, error handling, storage, APIs, testing, debugging, security basics, performance, and practical project development.

The syllabus is suitable for beginners, students, engineering graduates, working professionals, and developers who want to build a strong JavaScript foundation before moving to React, Angular, Vue, Node.js, Next.js, TypeScript, or full-stack development.

Course Objectives

By the end of this course, learners will be able to:

- Understand JavaScript fundamentals and execution behavior.
- Write, execute, debug, and maintain JavaScript programs.
- Work with variables, data types, operators, conditions, loops, arrays, and objects.
- Design reusable functions and modules.
- Manipulate HTML and CSS using the DOM.
- Handle browser events and user interactions.
- Use modern JavaScript features introduced in ES6 and later.
- Understand scope, closures, hoisting, and execution context.
- Apply object-oriented and functional programming concepts.
- Work with asynchronous JavaScript using callbacks, promises, and async/await.
- Fetch and process data from APIs.
- Use browser storage and common Web APIs.
- Handle errors and write robust JavaScript applications.
- Write unit tests for JavaScript code.
- Use Git and GitHub for version control.
- Build portfolio-ready JavaScript projects.
- Prepare for JavaScript interviews and coding assessments.

Module 1: Introduction to JavaScript

Topics

- Introduction to Programming
- What is JavaScript?
- History and Evolution of JavaScript
- JavaScript vs Java
- JavaScript and ECMAScript
- Role of JavaScript in Web Development
- Client-Side JavaScript
- Server-Side JavaScript Overview
- Applications of JavaScript
- Browser JavaScript Engines
- V8
- SpiderMonkey

- JavaScriptCore
- JavaScript Runtime Environment
- ECMAScript Versions Overview
- ES5
- ES6 / ES2015
- Modern JavaScript
- JavaScript Ecosystem Overview

Module 2: JavaScript Development Environment

Topics

- Running JavaScript in Browser
- Browser Developer Tools
- JavaScript Console
- `<script>` Tag
- Inline JavaScript
- Internal JavaScript
- External JavaScript
- Script Loading
- `defer`
- `async`
- Visual Studio Code
- Node.js for Running JavaScript Locally
- Browser vs Node.js Runtime
- Creating `.js` Files
- Running JavaScript from Terminal
- Code Formatting
- ESLint Overview
- Prettier Overview

Practical Exercises

- Hello World in Browser
- Hello World in Node.js
- External JavaScript File
- Browser Console Practice

Module 3: JavaScript Syntax and Program Structure

Topics

- JavaScript Statements
- Expressions
- Semicolons
- Automatic Semicolon Insertion
- Comments
- Single-Line Comments
- Multi-Line Comments
- Identifiers
- Keywords
- Case Sensitivity
- Code Blocks
- Strict Mode
- `"use strict"`
- Naming Conventions

- Readable Code
- JavaScript Style Guidelines

Module 4: Variables and Data Types

Topics

- Variables
- var
- let
- const
- Variable Declaration
- Variable Initialization
- Reassignment
- Scope Differences
- Primitive Data Types
- String
- Number
- BigInt
- Boolean
- Undefined
- Null
- Symbol
- Reference Types
- Objects
- Arrays
- Functions as Objects
- Dynamic Typing
- typeof
- Type Checking
- null Behavior
- Type Conversion
- Explicit Conversion
- Implicit Conversion
- Type Coercion

Practical Exercises

- Declare Variables with let and const
- Check Data Types
- Type Conversion Programs
- Swap Variables
- Basic User Data Program

Module 5: Operators and Expressions

Topics

- Arithmetic Operators
- Assignment Operators
- Comparison Operators
- Strict Equality ===
- Loose Equality ==
- Strict Inequality !==
- Logical Operators
- Unary Operators

- Increment and Decrement
- Exponentiation Operator
- Ternary Operator
- Nullish Coalescing Operator ??
- Optional Chaining ?.
- Logical Assignment Operators
- Bitwise Operators Overview
- Operator Precedence
- Expressions
- Truthy and Falsy Values

Practical Exercises

- Even or Odd
- Largest Number
- Leap Year
- Eligibility Checker
- Nullish Coalescing Examples

Module 6: Input and Output

Topics

- `console.log()`
- `console.error()`
- `console.warn()`
- `console.table()`
- Browser Alerts
- `alert()`
- `prompt()`
- `confirm()`
- Reading Input from HTML Forms
- Output to DOM
- Template Literals
- String Interpolation

Practical Exercises

- Greeting Application
- Calculator Input
- Student Details
- Confirmation Dialog Example

Module 7: Decision-Making Statements

Topics

- `if`
- `if-else`
- Nested `if`
- `else-if`
- `switch`
- `case`
- `default`
- Ternary Operator
- Logical Short-Circuiting
- Conditional Patterns

- Best Practices for Decision Logic

Practical Exercises

- Grade Calculator
- Positive, Negative, or Zero
- Day-of-Week Application
- Menu-Driven Calculator
- Discount Calculator

Module 8: Loops and Iteration

Topics

- for
- while
- do-while
- Nested Loops
- break
- continue
- for...of
- for...in
- Iterating Arrays
- Iterating Objects
- forEach()
- Loop Performance Basics

Practical Exercises

- Multiplication Table
- Factorial
- Fibonacci Series
- Prime Numbers
- Number Patterns
- Iterate an Array
- Iterate Object Properties

Module 9: Strings

Topics

- Creating Strings
- Single Quotes
- Double Quotes
- Template Literals
- String Immutability
- String Length
- Character Access
- Important String Methods
- charAt()
- includes()
- indexOf()
- lastIndexOf()
- startsWith()
- endsWith()
- slice()
- substring()

- `replace()`
- `replaceAll()`
- `split()`
- `trim()`
- `toUpperCase()`
- `toLowerCase()`
- `repeat()`
- String Concatenation
- Template Literals
- Multiline Strings
- String Comparison

Practical Exercises

- Reverse a String
- Palindrome Checker
- Count Vowels
- Word Counter
- Character Frequency
- Remove Extra Spaces

Module 10: Arrays

Topics

- Creating Arrays
- Array Indexing
- Updating Elements
- Array Length
- Nested Arrays
- Common Array Methods
- `push()`
- `pop()`
- `shift()`
- `unshift()`
- `slice()`
- `splice()`
- `concat()`
- `includes()`
- `indexOf()`
- `join()`
- `reverse()`
- `sort()`
- Copying Arrays
- Shallow Copy
- Multidimensional Arrays

Practical Exercises

- Sum and Average
- Maximum and Minimum
- Reverse an Array
- Remove Duplicates
- Sort Numbers
- Matrix Representation

Module 11: Advanced Array Methods

Topics

- `forEach()`
- `map()`
- `filter()`
- `reduce()`
- `find()`
- `findIndex()`
- `some()`
- `every()`
- `flat()`
- `flatMap()`
- `Array.from()`
- `Array.isArray()`
- Chaining Array Methods
- Functional Array Processing

Practical Exercises

- Filter Employees by Salary
- Transform Product Prices
- Calculate Total with `reduce()`
- Find Student by ID
- Flatten Nested Arrays

Module 12: Objects

Topics

- Object Literals
- Properties
- Methods
- Accessing Properties
- Dot Notation
- Bracket Notation
- Adding Properties
- Updating Properties
- Deleting Properties
- Nested Objects
- Object Methods
- `this` Keyword
- Property Shorthand
- Method Shorthand
- Computed Property Names
- Object Destructuring
- Object Spread
- Object Rest
- `Object.keys()`
- `Object.values()`
- `Object.entries()`
- `Object.assign()`
- Object Copying
- Shallow Copy

Practical Exercises

- Student Object
- Product Object
- Employee Object
- Nested Address Object
- Merge Objects

Module 13: Functions

Topics

- Function Declaration
- Function Expression
- Anonymous Functions
- Calling Functions
- Parameters
- Arguments
- Return Values
- Default Parameters
- Rest Parameters
- Spread Syntax
- Function Scope
- Block Scope
- Nested Functions
- Functions as Values
- Higher-Order Functions
- Callback Functions
- Pure Functions
- Recursive Functions

Practical Exercises

- Calculator Functions
- Factorial Using Recursion
- Fibonacci Using Recursion
- Higher-Order Function Example
- Callback Example

Module 14: Arrow Functions

Topics

- Arrow Function Syntax
- Single-Parameter Arrow Functions
- Multiple Parameters
- Implicit Return
- Block Body
- Arrow Functions with Array Methods
- Lexical `this`
- Arrow Functions vs Regular Functions
- Limitations of Arrow Functions

Practical Exercises

- Convert Traditional Functions to Arrow Functions
- Filter Data with Arrow Functions
- Sort Objects with Arrow Functions

Module 15: Scope, Hoisting, and Execution Context

Topics

- Global Scope
- Function Scope
- Block Scope
- Lexical Scope
- Scope Chain
- Variable Hoisting
- Function Hoisting
- var Hoisting
- let and const
- Temporal Dead Zone
- Execution Context
- Global Execution Context
- Function Execution Context
- Call Stack
- Variable Environment
- Lexical Environment Overview

Module 16: Closures

Topics

- What is a Closure?
- Lexical Environment
- Functions Returning Functions
- Private State with Closures
- Data Encapsulation
- Function Factories
- Closures in Event Handlers
- Common Closure Pitfalls
- Practical Use Cases

Practical Exercises

- Counter Using Closure
- Private Variable Example
- Function Factory
- Memoization Introduction

Module 17: Destructuring, Rest, and Spread

Topics

- Array Destructuring
- Object Destructuring
- Default Values
- Nested Destructuring
- Renaming Variables
- Rest Parameters
- Rest Properties
- Spread with Arrays
- Spread with Objects
- Cloning
- Merging

- Function Calls with Spread

Practical Exercises

- Swap Values Using Destructuring
- Extract Object Data
- Merge Arrays
- Clone Objects

Module 18: Object-Oriented Programming in JavaScript

Topics

- OOP Concepts
- Objects and Prototypes
- Constructor Functions
- new Keyword
- Prototype Chain
- Prototype Inheritance
- ES6 Classes
- Class Syntax
- Constructor
- Instance Methods
- Static Methods
- Fields
- Private Fields
- Getters and Setters
- Inheritance
- extends
- super
- Method Overriding
- Encapsulation
- Polymorphism
- Composition vs Inheritance

Practical Exercises

- Student Class
- Employee Class
- Bank Account Class
- Shape Hierarchy

Module 19: Prototypes and Prototype Chain

Topics

- Prototype-Based Inheritance
- prototype
- __proto__ Concept
- Object.getPrototypeOf()
- Prototype Chain
- Constructor Functions
- Shared Methods
- Prototype Lookup
- Built-In Prototypes
- Extending Built-In Prototypes: Why to Avoid
- Classes as Syntax over Prototypes

Module 20: Error Handling

Topics

- Runtime Errors
- Syntax Errors
- Logical Errors
- try
- catch
- finally
- throw
- Error Object
- Error Types
- Error
- TypeError
- ReferenceError
- SyntaxError
- RangeError
- Custom Error Classes
- Error Propagation
- Defensive Programming
- Error Handling Best Practices

Practical Exercises

- Input Validation Error
- Custom Age Error
- API Error Handling
- Calculator Error Handling

Module 21: DOM Fundamentals

Topics

- What is the DOM?
- DOM Tree
- Document Object
- Selecting Elements
- getElementById()
- getElementsByName()
- getElementsByTagName()
- querySelector()
- querySelectorAll()
- Reading Content
- Updating Content
- textContent
- innerHTML
- Attributes
- Classes
- Inline Styles
- Creating Elements
- Appending Elements
- Removing Elements
- Replacing Elements
- Traversing the DOM

Practical Exercises

- Change Heading Text
- Dynamic List Creation
- Add and Remove Elements
- Change Styles Dynamically

Module 22: Events

Topics

- Browser Events
- Event Listeners
- `addEventListener()`
- Click Events
- Keyboard Events
- Mouse Events
- Form Events
- Input Events
- Change Events
- Submit Events
- Event Object
- Event Target
- `preventDefault()`
- Event Bubbling
- Event Capturing
- Event Delegation
- Removing Event Listeners

Practical Exercises

- Button Click Counter
- Keyboard Tracker
- Form Submit Handler
- Event Delegation with List Items

Module 23: Forms and Validation

Topics

- Accessing Form Elements
- Input Values
- Form Submission
- Preventing Default Submission
- Required Fields
- Email Validation
- Password Validation
- Numeric Validation
- Regular Expressions
- Error Messages
- Client-Side Validation
- Custom Validation
- Form Data API Overview

Practical Exercises

- Registration Form
- Login Validation
- Contact Form

- Password Strength Checker

Module 24: Browser Object Model

Topics

- window
- document
- navigator
- location
- history
- screen
- Browser Navigation
- URL Information
- Timers
- setTimeout()
- setInterval()
- clearTimeout()
- clearInterval()

Practical Exercises

- Digital Clock
- Countdown Timer
- Redirect Application
- Browser Information Display

Module 25: Local Storage, Session Storage, and Cookies

Topics

- Web Storage API
- localStorage
- sessionStorage
- setItem()
- getItem()
- removeItem()
- clear()
- Storing Objects with JSON
- Cookies Overview
- Cookie Basics
- Storage Differences
- Security Considerations
- Storage Limits Overview

Practical Exercises

- Save Theme Preference
- Persistent To-Do List
- Store User Settings
- Session-Based Form Data

Module 26: JSON

Topics

- What is JSON?
- JSON Syntax

- JavaScript Objects vs JSON
- `JSON.stringify()`
- `JSON.parse()`
- Nested JSON
- Arrays in JSON
- Working with API Responses
- Error Handling During Parsing

Practical Exercises

- Convert Object to JSON
- Parse JSON Data
- Store Object in Local Storage
- Process Nested JSON

Module 27: Asynchronous JavaScript Fundamentals

Topics

- Synchronous vs Asynchronous Execution
- JavaScript Runtime
- Call Stack
- Web APIs
- Callback Queue
- Event Loop
- Microtask Queue
- Macrotasks
- Timers
- Callback Functions
- Nested Callbacks
- Callback Hell
- Error-First Callback Concept

Module 28: Promises

Topics

- Promise Concept
- Promise States
- Pending
- Fulfilled
- Rejected
- Creating Promises
- `resolve`
- `reject`
- `.then()`
- `.catch()`
- `.finally()`
- Promise Chaining
- Error Propagation
- `Promise.all()`
- `Promise.allSettled()`
- `Promise.race()`
- `Promise.any()`

Practical Exercises

- Simulated API Request
- Promise Chaining
- Run Multiple Promises
- Promise Error Handling

Module 29: Async and Await

Topics

- async Functions
- await
- Promise-Based Async Code
- Sequential Execution
- Parallel Execution
- Error Handling with try-catch
- Async Functions Returning Promises
- Awaiting Multiple Operations
- Common Async Pitfalls
- Async Best Practices

Practical Exercises

- Async Data Fetch Simulation
- Parallel Async Tasks
- Error Handling in Async Functions

Module 30: Fetch API and REST API Consumption

Topics

- Introduction to HTTP
- REST API Basics
- HTTP Methods
- GET
- POST
- PUT
- PATCH
- DELETE
- Fetch API
- fetch()
- Request Options
- Headers
- JSON Responses
- HTTP Status Codes
- Error Handling
- Sending JSON Data
- Query Parameters
- URLSearchParams Overview
- CORS Overview
- API Security Basics

Practical Exercises

- Fetch User Data
- Display API Data in DOM
- POST Form Data
- Build CRUD Frontend Simulation

Module 31: JavaScript Modules

Topics

- Why Modules?
- ES Modules
- `export`
- Named Exports
- Default Exports
- `import`
- Import Aliases
- Module Scope
- Dynamic Imports
- Browser Module Scripts
- CommonJS Overview
- ES Modules vs CommonJS
- Organizing Application Code

Practical Exercises

- Utility Module
- Calculator Module
- Multi-File JavaScript Project

Module 32: Regular Expressions

Topics

- Regular Expression Syntax
- Regex Literals
- RegExp
- Character Classes
- Quantifiers
- Groups
- Anchors
- Flags
- `test()`
- `exec()`
- String Methods with Regex
- Search and Replace
- Validation Patterns

Practical Exercises

- Email Validation
- Mobile Number Validation
- Password Validation
- Extract Numbers from Text

Module 33: Date and Time

Topics

- Date Object
- Creating Dates
- Current Date and Time
- Getters
- Setters

- Timestamps
- Date Formatting
- Date Comparison
- Date Arithmetic
- Time Zones Overview
- Intl.DateTimeFormat
- Internationalization Basics

Practical Exercises

- Age Calculator
- Date Difference
- Countdown Timer
- Formatted Date Display

Module 34: Math and Number Utilities

Topics

- Math Object
- Math.round()
- Math.floor()
- Math.ceil()
- Math.random()
- Math.max()
- Math.min()
- Math.pow()
- Math.sqrt()
- Number
- Number.isInteger()
- Number.isNaN()
- parseInt()
- parseFloat()
- Floating-Point Precision
- BigInt Overview

Practical Exercises

- Random Number Generator
- Dice Simulator
- OTP Generator
- Number Validation

Module 35: Set and Map

Topics

- Set
- Creating Sets
- Adding Values
- Removing Values
- Unique Values
- Iterating Sets
- Map
- Key-Value Pairs
- Object Keys vs Map Keys
- Adding Entries

- Reading Entries
- Removing Entries
- Iterating Maps
- WeakSet Overview
- WeakMap Overview

Practical Exercises

- Remove Duplicates
- Unique Visitor Tracker
- Student Marks Map
- Object Metadata with WeakMap Overview

Module 36: Iterators and Generators**Topics**

- Iterable Protocol
- Iterator Protocol
- `Symbol.iterator`
- Built-In Iterables
- Custom Iterators
- Generator Functions
- `function*`
- `yield`
- Generator Objects
- Lazy Evaluation
- Infinite Generators
- `yield*`

Practical Exercises

- Custom Range Iterator
- Fibonacci Generator
- Infinite ID Generator

Module 37: Symbols**Topics**

- `Symbol`
- Unique Property Keys
- Global Symbol Registry
- `Symbol.for()`
- `Symbol.keyFor()`
- Well-Known Symbols
- `Symbol.iterator`
- Use Cases for Symbols

Module 38: Advanced Functions**Topics**

- First-Class Functions
- Higher-Order Functions
- Closures
- Function Composition
- Currying Overview

- Partial Application
- Memoization
- IIFE
- Function Binding
- `call()`
- `apply()`
- `bind()`
- `this` Behavior
- Method Borrowing

Practical Exercises

- Memoized Function
- Function Composition
- Bound Event Handler
- Custom Partial Function

Module 39: Deep Dive into `this`

Topics

- Global `this`
- `this` in Functions
- `this` in Methods
- `this` in Arrow Functions
- `this` in Event Handlers
- Constructor Context
- `call()`
- `apply()`
- `bind()`
- Strict Mode Behavior
- Common `this` Pitfalls

Module 40: Functional Programming Concepts

Topics

- Pure Functions
- Immutability
- Higher-Order Functions
- Function Composition
- Declarative Programming
- Side Effects
- Map-Filter-Reduce Pattern
- Referential Transparency Overview
- Functional vs Imperative Style

Module 41: Browser APIs

Topics

- Clipboard API Overview
- Geolocation API Overview
- History API
- URL API
- `URLSearchParams`
- Notification API Overview

- Intersection Observer Overview
- Mutation Observer Overview
- Drag and Drop API Overview
- File API Overview
- Web Workers Overview

Practical Exercises

- Copy Text to Clipboard
- Read Query Parameters
- Lazy Loading Simulation
- File Preview

Module 42: Web Workers and Background Processing

Topics

- JavaScript Single-Threaded Model
- Web Workers
- Creating a Worker
- Sending Messages
- Receiving Messages
- Worker Limitations
- Use Cases
- Background Computation
- Terminating Workers

Module 43: Debugging JavaScript

Topics

- Browser DevTools
- Console
- Breakpoints
- Conditional Breakpoints
- Step Into
- Step Over
- Step Out
- Watch Expressions
- Call Stack
- Scope Inspection
- Network Tab Overview
- Sources Tab
- Debugger Statement
- Common JavaScript Errors
- ReferenceError
- TypeError
- SyntaxError
- RangeError

Module 44: Testing JavaScript

Topics

- Importance of Testing
- Unit Testing
- Integration Testing Overview

- Test Cases
- Assertions
- Arrange-Act-Assert
- Jest Overview
- Vitest Overview
- Test Suites
- Test Hooks
- Testing Functions
- Testing Async Code
- Mocking Overview
- Test-Driven Development Introduction

Practical Exercises

- Test Calculator Functions
- Test String Utilities
- Test Async Function
- Test Array Utilities

Module 45: JavaScript Security Basics

Topics

- Cross-Site Scripting Overview
- Unsafe `innerHTML`
- Input Validation
- Output Encoding
- Content Security Policy Overview
- CORS Basics
- Same-Origin Policy Overview
- Avoiding `eval()`
- Secure Storage Considerations
- Secrets in Frontend Code
- Dependency Security Awareness
- Safe API Usage

Module 46: Performance Optimization Basics

Topics

- Avoiding Unnecessary DOM Updates
- DOM Reflow and Repaint Overview
- Event Delegation
- Debouncing
- Throttling
- Lazy Loading
- Efficient Loops
- Efficient Array Methods
- Memory Leaks
- Detached DOM Nodes
- Timer Cleanup
- Caching Values
- Performance API Overview

Practical Exercises

- Debounced Search

- Throttled Scroll Handler
- Optimize DOM Updates

Module 47: Package Management and npm Basics

Topics

- Introduction to Node.js
- npm
- npm Registry
- package.json
- npm init
- Installing Packages
- Development Dependencies
- Production Dependencies
- node_modules
- package-lock.json
- npm Scripts
- Semantic Versioning Overview
- Updating Packages
- Removing Packages
- Security Auditing Overview

Module 48: Build Tools and Modern JavaScript Tooling Overview

Topics

- Why Build Tools?
- Module Bundlers
- Vite Overview
- Webpack Overview
- Babel Overview
- Transpilation
- Bundling
- Minification
- Source Maps
- Development Server
- Environment Variables Overview
- Modern Frontend Toolchain

Module 49: Clean Code and Professional JavaScript Practices

Topics

- Naming Conventions
- const by Default
- Avoiding Global Variables
- Small Functions
- Separation of Concerns
- DRY Principle
- KISS Principle
- Immutability Where Appropriate
- Pure Functions
- Error Handling
- Async Error Handling
- Modular Code

- ESLint
- Prettier
- JSDoc Overview
- Code Reviews
- Consistent Style
- Avoiding Deep Nesting

Module 50: Problem-Solving and Algorithmic Thinking

Topics

- Understanding Problem Statements
- Identifying Inputs and Outputs
- Constraints
- Pseudocode
- Dry Runs
- Edge Cases
- Time Complexity Introduction
- Space Complexity Introduction
- Big-O Notation Basics
- Recursion
- Searching
- Sorting
- Common Coding Patterns
- Choosing Appropriate Data Structures

Practice Categories

- Number Problems
- String Problems
- Array Problems
- Object Problems
- Recursion Problems
- DOM Problems
- Async Problems
- API Problems

Module 51: Data Structures Using JavaScript

Topics

- Arrays
- Objects
- Sets
- Maps
- Stacks
- Queues
- Linked List Concepts
- Trees Overview
- Binary Search Tree Introduction
- Graphs Overview
- Hashing Concepts
- Choosing Appropriate Structures

Algorithms

- Linear Search

- Binary Search
- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort Overview
- Quick Sort Overview

Module 52: Git and GitHub for JavaScript Developers

Topics

- Introduction to Version Control
- Installing Git
- Creating a Repository
- `git init`
- `git add`
- `git commit`
- `git status`
- `git log`
- Branching Basics
- Merging
- GitHub Repository Creation
- Push and Pull
- Pull Requests Overview
- README Documentation
- `.gitignore`
- Ignoring `node_modules`
- Building a Programming Portfolio

Module 53: AI-Assisted JavaScript Development

Topics

- Using AI Coding Assistants Responsibly
- Generating Boilerplate Code
- Explaining JavaScript Code
- Understanding Errors
- Debugging with AI Assistance
- Generating Unit Tests
- Refactoring Suggestions
- DOM Code Review
- Async/Await Review
- API Integration Assistance
- Security Review
- Performance Suggestions
- Generating Documentation
- Risks of Blindly Using Generated Code
- Validating AI-Generated Solutions
- Prompting Techniques for JavaScript Programming

Recommended Practice

Students should learn to:

- Ask AI for explanations instead of only solutions.
- Verify generated code manually.
- Test edge cases.

- Inspect browser behavior.
- Review async error handling.
- Avoid exposing secrets in frontend code.
- Validate security-sensitive suggestions.
- Understand every generated line before submission.

Module 54: Mini Projects

Students should complete multiple mini projects such as:

- Calculator
- Digital Clock
- Countdown Timer
- To-Do List
- Quiz Application
- Expense Tracker
- Notes Application
- Weather Application Using API
- Movie Search Application
- Form Validation Application
- Password Generator
- Image Gallery
- Shopping Cart
- Currency Converter
- GitHub Profile Finder
- Recipe Finder
- Book Search Application
- Local Storage Task Manager

Module 55: Capstone Project

Learners should build one complete JavaScript application using the concepts covered in the course.

Suggested Capstone Projects

- Expense Management Application
- E-Commerce Frontend
- Task Management Application
- Student Management System
- Weather Dashboard
- Movie Discovery Application
- Personal Finance Tracker
- Quiz and Assessment Platform
- Inventory Management Frontend
- API-Based Admin Dashboard

Capstone Requirements

The project should demonstrate:

- Modular JavaScript
- Functions
- Arrays and objects
- DOM manipulation
- Events
- Form validation
- ES6+ features

- Async/await
- API integration
- Error handling
- Local storage where appropriate
- Responsive interaction
- Clean code structure
- Testing basics
- Git-based version control
- README documentation

Module 56: JavaScript Interview Preparation

Core JavaScript Interview Topics

- var vs let vs const
- Primitive vs Reference Types
- Type Coercion
- == vs ===
- Scope
- Hoisting
- Temporal Dead Zone
- Execution Context
- Call Stack
- Closures
- Higher-Order Functions
- Callback Functions
- Arrow Functions
- this
- call, apply, and bind
- Objects
- Prototype Chain
- Prototypal Inheritance
- Classes
- Destructuring
- Spread and Rest
- Array Methods
- Map, Filter, Reduce
- Event Loop
- Microtasks vs Macrotasks
- Promises
- Async/Await
- DOM
- Event Bubbling
- Event Capturing
- Event Delegation
- Local Storage vs Session Storage
- JSON
- ES Modules
- CommonJS Overview
- Shallow vs Deep Copy
- Debouncing
- Throttling
- Memoization
- Fetch API

- CORS
- Error Handling
- Memory Leaks
- ES6+ Features

Interview Activities

- Conceptual Questions
- Output-Based Questions
- Debugging Questions
- Coding Challenges
- Array and Object Problems
- Closure Questions
- Async JavaScript Questions
- DOM Problems
- API Problems
- Mock Interviews
- Timed Coding Tests

Recommended Lab Assignments

Learners should complete at least:

- 20 Basic Syntax and Operator Programs
- 20 Conditional and Looping Programs
- 20 String and Array Exercises
- 20 Object and Function Exercises
- 15 ES6+ Exercises
- 15 DOM Exercises
- 10 Event Handling Exercises
- 10 Form Validation Exercises
- 15 Async and Promise Exercises
- 10 API Integration Exercises
- 10 OOP and Prototype Exercises
- 5 Testing Exercises
- 5 Mini Projects
- 1 Capstone Project

Suggested Course Duration

- Section — Suggested Duration
- JavaScript Fundamentals — 12–15 Hours
- Control Flow, Strings, Arrays, and Objects — 20–25 Hours
- Functions, Scope, Closures, and OOP — 18–22 Hours
- ES6+ and Modern JavaScript — 15–18 Hours
- DOM, Events, Forms, and Browser APIs — 20–25 Hours
- Async JavaScript, Promises, and APIs — 15–18 Hours
- Modules, Tooling, Testing, and Debugging — 12–15 Hours
- Security, Performance, and Clean Code — 8–10 Hours
- Problem Solving and Data Structures — 12–15 Hours
- Projects and Interview Preparation — 20–25 Hours

Recommended Total Duration: approximately **150–188 hours**, depending on practical depth, assignments, API work, and project complexity.

Prerequisites

No prior programming experience is required.

Basic knowledge of HTML and CSS is recommended for DOM, events, and browser-based project modules.

Basic familiarity with computers, files, folders, and command-line usage is helpful.

Tools and Technologies

- JavaScript
- ECMAScript
- HTML
- CSS
- Browser Developer Tools
- Visual Studio Code
- Node.js
- npm
- Vite
- ESLint
- Prettier
- Jest / Vitest Overview
- Git
- GitHub
- REST APIs
- JSON
- Local Storage
- Command Line / Terminal

Learning Outcomes

After completing the course, learners should be able to:

- Write structured and modern JavaScript applications.
- Understand JavaScript data types, scope, functions, closures, and objects.
- Manipulate the DOM and handle user interactions.
- Apply ES6+ features effectively.
- Work with asynchronous JavaScript.
- Consume REST APIs using Fetch.
- Use promises and async/await correctly.
- Organize applications using modules.
- Understand prototypal inheritance and classes.
- Debug and test JavaScript applications.
- Use browser storage and common Web APIs.
- Apply basic security and performance practices.
- Use Git and GitHub professionally.
- Build portfolio-ready JavaScript projects.
- Solve JavaScript interview and coding problems.
- Progress confidently to modern frontend or backend frameworks.

Recommended Progression After JavaScript

After completing JavaScript, learners can continue with:

- TypeScript
- React.js
- Angular
- Vue.js

- Node.js
- Express.js
- Next.js
- Full-Stack JavaScript Development
- MERN Stack Development
- MEAN Stack Development
- REST API Development
- GraphQL
- Testing with Jest and Playwright
- Web Performance Optimization
- Progressive Web Applications
- React Native
- Electron
- Data Structures and Algorithms in JavaScript
- Frontend System Design
- Full-Stack System Design

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.